МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

федеральное государственное автономное
образовательное учреждение высшего образования
«Самарский национальный исследовательский университет
имени академика С.П. Королева»
(Самарский университет)

ЛАБОРАТОРНАЯ РАБОТА № 3

«Работа с исключениями и интерфейсами в наборе классов табулированной функции»

по курсу
Объектно-ориентированное программирование

Выполнил: Волынец Никита,
6203-010302D

Оглавление

Задание №1.....3
Задание №2..... 3
Задание №3..... 4-6
Задание №4..... 6-7
Задание №5..... 7
Задание №6.....8

Задание №1

Я разработал два специализированных класса исключений. FunctionPointIndexOutOfBoundsException наследуется от IndexOutOfBoundsException и обрабатывает попытки доступа к несуществующим индексам точек. InappropriateFunctionPointException наследуется от Exception и реагирует на нарушения логической целостности данных - добавление точек с одинаковыми координатами X или нарушение порядка сортировки. Оба класса содержат конструкторы с возможностью передачи детализированных сообщений об ошибках.

```java
package functions;

// исключение выхода за границы точек
public class FunctionPointIndexOutOfBoundsException extends IndexOutOfBoundsException{ 42 usages
    // конструктор по умолчанию
    public FunctionPointIndexOutOfBoundsException() { 7 usages
        // вызываем конструктор родителя
        super();
    }
    // конструктор с сообщением
    public FunctionPointIndexOutOfBoundsException(String mes) { 22 usages
        // передаем сообщение родителю
        super(mes);
    }
}
```

```java
package functions;

// исключение неподходящей точки
public class InappropriateFunctionPointException extends Exception{ 17 usages
    // конструктор по умолчанию
    public InappropriateFunctionPointException(){ 8 usages
        // вызываем конструктор родителя
        super();
    }
    // конструктор с сообщением
    public InappropriateFunctionPointException(String mes){ 14 usages
        // передаем сообщение родителю
        super(mes);
    }
}
```

Задание №2

Я провел комплексную доработку класса, добавив систему проверок во все критические методы. Конструкторы теперь валидируют входные параметры - проверяют корректность интервала и минимальное количество точек. Методы работы с точками контролируют границы массива и целостность данных. Особое внимание уделено сохранению упорядоченности точек при модификациях. При обнаружении нарушений выбрасываются соответствующие исключения с информативными сообщениями.

Задание №3

При реализации класса LinkedListTabulatedFunction я использовал архитектуру двусвязного циклического списка с выделенной головой. Внутренний класс FunctionNode был объявлен как static class с модификатором доступа private, что обеспечивает полную инкапсуляцию - внешний код не имеет прямого доступа к узлам списка. Каждый узел содержит три поля: объект FunctionPoint для хранения данных, а также ссылки prev и next на соседние узлы. Для управления доступом к полям реализованы геттеры и сеттеры.

```
private static class FunctionNode{  25 usages
    private FunctionPoint point; // точка функции  3 usages
    private FunctionNode prev; // ссылка на предыдущий узел  3 usages
    private FunctionNode next; // ссылка на следующий узел  3 usages

    public FunctionNode(FunctionPoint point){  5 usages
        this.point = point; // устанавливаем точку
        this.prev = null; // предыдущий узел пока null
        this.next = null; // следующий узел пока null
    }

    public FunctionPoint getPoint(){  20 usages
        return point; // возвращаем точку
    }
    public void setPoint(FunctionPoint point){  1 usage
        this.point = point; // устанавливаем новую точку
    }
    public FunctionNode getPrev(){  4 usages
        return prev; // возвращаем предыдущий узел
    }
    public void setPrev(FunctionNode prev){  7 usages
        this.prev = prev; // устанавливаем предыдущий узел
    }
    public FunctionNode getNext(){  2 usages
        return next; // возвращаем следующий узел
    }
    public void setNext(FunctionNode next){  7 usages
        this.next = next; // устанавливаем следующий узел
    }
}
```

Основной класс LinkedListTabulatedFunction содержит поле head. В пустом состоянии head ссылается сам на себя. Поле pointcount отслеживает количество элементов, что исключает необходимость пересчета длины списка при каждом обращении.

```
public class LinkedListTabulatedFunction implements TabulatedFunction{  1 usage
    private FunctionNode head = new FunctionNode( point: null); // голова списка  14 usages
    private int pointcount; // количество точек  28 usages
```

Метод getNodeByIndex реализован с оптимизацией доступа - он анализирует, с какой стороны ближе идти к целевому индексу (от начала или от конца списка), что значительно ускоряет операции в больших списках. Для добавления элементов разработаны два метода: addNodeToTail для вставки в конец и addNodeByIndex для вставки в произвольную позицию. Оба метода корректно обновляют связи между узлами и увеличивают счетчик элементов.

```
public FunctionNode getNodeByIndex(int index) throws FunctionPointIndexOutOfBoundsException{
    if(index < 0 || index >= pointcount){
        throw new FunctionPointIndexOutOfBoundsException("Индекс не входит в границы");
    }
    FunctionNode node = head; // начинаем с головы
    if(index < pointcount/2){
        for(int i =0; i<=index; i++){
            node = node.getNext(); // идем вперед по списку
        }
    }
    else{
        for(int i =pointcount-1; i >=index; i--){
            node = node.getPrev(); // идем назад по списку
        }
    }
    return node; // возвращаем найденный узел
}
public FunctionNode addNodeToTail(FunctionPoint point){  3 usages
    FunctionNode newNode = new FunctionNode(point); // создаем новый узел
    FunctionNode tail = head.getPrev(); // получаем хвост списка

    newNode.setPrev(tail); // новый узел указывает на хвост
    newNode.setNext(head); // новый узел указывает на голову
    tail.setNext(newNode); // хвост указывает на новый узел
    head.setPrev(newNode); // голова указывает на новый узел

    pointcount ++; // увеличиваем счетчик точек
    return newNode; // возвращаем новый узел
}
public FunctionNode addNodeByIndex(int index, FunctionPoint point) throws FunctionPointIndexOutOfBoundsException{
    if(index < 0 || index > pointcount){
        throw new FunctionPointIndexOutOfBoundsException("Индекс не входит в границы");
    }
    if(index == pointcount){
        return addNodeToTail(point); // добавляем в конец
    }

    FunctionNode curNode = getNodeByIndex(index); // получаем текущий узел
    FunctionNode newNode = new FunctionNode(point); // создаем новый узел
    FunctionNode prevNode = curNode.getPrev(); // получаем предыдущий узел

    newNode.setPrev(prevNode); // новый узел указывает на предыдущий
    newNode.setNext(curNode); // новый узел указывает на текущий
    prevNode.setNext(newNode); // предыдущий указывает на новый
    curNode.setPrev(newNode); // текущий указывает на новый

    pointcount++; // увеличиваем счетчик точек
    return newNode; // возвращаем новый узел
}
```

Метод deleteNodeByIndex обеспечивает безопасное удаление узлов - перед удалением он сохраняет ссылки на соседние узлы, затем перелинковывает их,

исключая удаляемый элемент из цепочки, и уменьшает счетчик. Все методы работы с узлами проверяют корректность индексов и выбрасывают исключения при выходе за границы допустимого диапазона.

```
public FunctionNode deleteNodeByIndex(int index) throws FunctionPointIndexOutOfBoundsException{  1 usage
    if(index < 0 || index >= pointcount){
        throw new FunctionPointIndexOutOfBoundsException("Индекс не входит в границы");
    }
    FunctionNode delNode = getNodeByIndex(index); // узел для удаления
    FunctionNode prevNode = delNode.getPrev(); // предыдущий узел
    FunctionNode nextNode = delNode.getNext(); // следующий узел

    prevNode.setNext(nextNode); // предыдущий указывает на следующий
    nextNode.setPrev(prevNode); // следующий указывает на предыдущий

    pointcount--; // уменьшаем счетчик точек
    return delNode; // возвращаем удаленный узел
}
```

Такая архитектура обеспечивает эффективное выполнение операций добавления и удаления элементов в любой позиции списка, а оптимизированный доступ по индексу делает работу с большими наборами данных достаточно производительно

Задание №4

Я полностью реализовал в LinkedListTabulatedFunction все методы, объявленные в интерфейсе TabulatedFunction. Конструкторы создают функцию с равномерным распределением точек или заданными значениями. Все публичные методы используют внутренние операции со списком, обеспечивая инкапсуляцию данных. Система исключений полностью соответствует ArrayTabulatedFunction, гарантируя одинаковое поведение обеих реализаций.

```java
package functions;

public interface TabulatedFunction {  8 usages  2 implementations
    // получить левую границу области определения функции
    public double getLeftDomainBorder();  2 usages  2 implementations

    // получить правую границу области определения функции
    public double getRightDomainBorder();  2 usages  2 implementations

    // вычислить значение функции в заданной точке x
    public double getFunctionValue(double x);  3 usages  2 implementations

    // получить общее количество точек в функции
    public int getPointsCount();  6 usages  2 implementations

    // получить копию точки по указанному индексу
    public FunctionPoint getPoint(int index) throws FunctionPointIndexOutOfBoundsException;  1 usage

    // заменить точку по указанному индексу
    public void setPoint(int index, FunctionPoint point) throws FunctionPointIndexOutOfBoundsExcep

    // получить координату x точки по индексу
    public double getPointX(int index) throws FunctionPointIndexOutOfBoundsException;  6 usages  2 imp

    // установить новую координату x для точки по индексу
    public void setPointX(int index, double x) throws FunctionPointIndexOutOfBoundsException, Inap

    // получить координату y точки по индексу
    public double getPointY(int index) throws FunctionPointIndexOutOfBoundsException;  5 usages  2 imp

    // установить новую координату y для точки по индексу
    public void setPointY(int index, double y) throws FunctionPointIndexOutOfBoundsException;  1 us

    // удалить точку по указанному индексу
    public void deletePoint(int index) throws FunctionPointIndexOutOfBoundsException, IllegalState

    // добавить новую точку в функцию
    public void addPoint(FunctionPoint point) throws InappropriateFunctionPointException;  3 usages
}
```

Пункт №5

Я провел рефакторинг архитектуры проекта, выделив общий интерфейс TabulatedFunction. Существующий класс TabulatedFunction был переименован в ArrayTabulatedFunction. Обе реализации теперь имплементируют единый интерфейс, что позволяет использовать их полиморфно. Такой подход обеспечивает гибкость системы и возможность легкого расширения.

Задание №6

Я разработал комплексную систему тестирования в классе Main. Тесты покрывают все основные сценарии работы: создание функций, модификацию точек, вычисление значений, интерполяцию. Отдельный метод testExceptions() проверяет корректность работы системы исключений. Тестирование подтвердило идентичность поведения обеих реализаций и правильную обработку всех ошибочных ситуаций.

```
Квадратичная функция y = 2,5*x:
(0,00, 0,00) (2,50, 6,25) (5,00, 12,50) (7,50, 18,75) (10,00, 25,00)
После добавления точек:
(0,00, 0,00) (1,50, 3,75) (2,50, 6,25) (3,50, 8,75) (5,00, 12,50) (7,50, 18,75) (10,00, 25,00)
После удаления точки:
(0,00, 0,00) (1,50, 3,75) (2,50, 6,25) (5,00, 12,50) (7,50, 18,75) (10,00, 25,00)
После изменения точки:
(0,00, 0,00) (1,50, 3,75) (3,00, 4,00) (5,00, 12,50) (7,50, 18,75) (10,00, 25,00)
Интерполяция:
f(0,50) = 1,25
f(1,20) = 3,00
f(2,50) = 3,92
f(3,20) = 4,85
f(6,80) = 17,00
Область определения: [0,00, 10,00]
Вне области определения:
f(-1) = NaN
f(11) = NaN
Финальное состояние:
(0,00, 0,00) (1,50, 3,75) (3,00, 4,00) (5,00, 12,50) (7,50, 18,75) (10,00, 25,00)

Тестирование исключений:
1. Попытка создать функцию с неправильными границами:
Поймано IllegalArgumentException: Левая граница больше левой или количество точек меньше 2
2. Попытка получить точку с неверным индексом:
Поймано FunctionPointIndexOutOfBoundsException: Индекс не входит в границы
3. Попытка нарушить порядок точек:
Поймано InappropriateFunctionPointException: Точка не входит в интервал
4. Попытка добавить точку с существующим X:
Поймано InappropriateFunctionPointException: Совпадают X
5. Попытка удалить слишком много точек:
Поймано IllegalStateException: Количество точек меньще 3
Тестирование исключений завершено

Process finished with exit code 0
```